Homography & Perspective Warp

Week 11 — Session 1: Transforms & the Homography Model

Practical Image Processing

Lecture 11.1 ▪ 60–70 min

Where we are

- Wk10: we matched the same points across two views of a scene.
- This session: the *maths of moving pixels* — the family of 2D transforms, and the one that models a camera: the **homography**.
- Next session we *use* it: warp an angled photo of a meter/document into a flat, front-on scan.

Two-session arc

S1 (today): the models — affine, projective, homogeneous coords.

S2: the pipeline — rectify, interpolate, RANSAC, DocScan.

Roadmap

Positions, not values

The transform hierarchy

Homogeneous coordinates

The homography

In-class exercise

Pitfalls & good practice

Wrap-up

Positions, not values

What is a geometric transform?

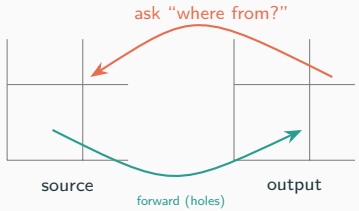
- So far we changed pixel *values* (brightness, edges). Now we change pixel *positions*.
- A geometric transform maps each output pixel (x', y') back to a source location (x, y).
- Examples: translate, rotate, scale, shear, and — the big one — **perspective**.

Key idea

Value operations answer "what colour is here?" Geometric operations answer "where does this point go?"

Forward vs. inverse mapping

- **Forward**: push each source pixel to its destination. Leaves holes and overlaps.
- **Inverse**: for each *output* pixel, ask where it came from in the source, then sample.
- Real warpers use inverse mapping — every output pixel gets exactly one value.



Note

Inverse mapping is why `cv2.warpPerspective` takes the transform *output*→*input* internally.

The transform hierarchy

A hierarchy of 2D transforms

Transform	Preserves	DoF	Matrix
Rigid (rotate+translate)	lengths, angles	3	2×3
Similarity (+scale)	angles, ratios	4	2×3
Affine	parallelism	6	2×3
Projective (homography)	straight lines only	8	3×3

Key idea

Each row is more general. A photo taken at an angle needs the most general one: a **homography**.

The hierarchy, visually



Left to right: fewer things kept fixed. The unit square becomes a rotated square, a scaled one, a parallelogram, and finally an arbitrary (but still straight-edged) quadrilateral.

Rigid & similarity in matrix form

Math

Rotation by θ then translation t:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}}_{R} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

Similarity multiplies R by a scalar scale s (4 DoF: θ, s, t_x, t_y).

- R is orthonormal: columns are unit length and perpendicular $\Rightarrow$ no stretching or shear.
- That is exactly why rigid motion keeps lengths and angles.

Affine transform

- 6 degrees of freedom: a 2×3 matrix. Maps parallelograms to parallelograms.
- `cv2.getAffineTransform(3 pts, 3 pts)`, then `cv2.warpAffine`.
- Good for rotation/scale/shear — but *cannot* model a page tilted away from the camera.

Math

Affine: $\begin{pmatrix} x' \\ y' \end{pmatrix} = A \begin{pmatrix} x \\ y \end{pmatrix} + t$, with $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. Straight, parallel lines stay parallel.

Worked example: a shear + scale

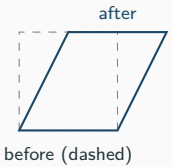
Math

$$A = \begin{pmatrix} 1 & 0.5 \\ 0 & 1 \end{pmatrix}, \; t = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

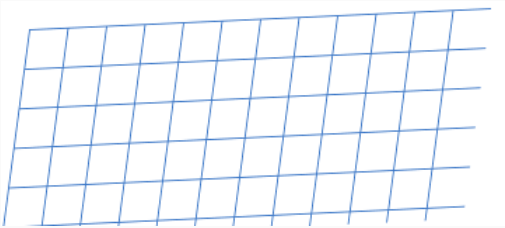
Map the unit square's corners:

- $(0, 0) \to (0, 0)$
- $(1, 0) \to (1, 0)$
- $(0, 1) \to (0.5, 1)$
- $(1, 1) \to (1.5, 1)$

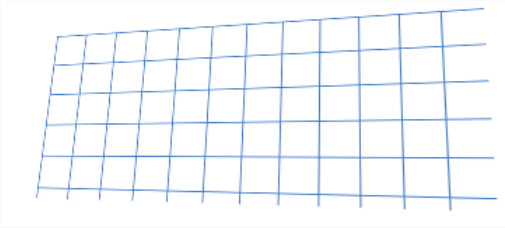
Bottom edge fixed, top edge slid right by $0.5\,y$: a horizontal shear.



Affine vs perspective — on a grid



affine: parallel lines stay parallel



perspective: parallel lines converge

The converging lines are the “vanishing point” effect of a real camera. Only a homography models it.

Homogeneous coordinates

Why homogeneous coordinates?

- Affine needs a matrix *plus* an added vector t — clumsy to chain.
- Trick: append a 1. A point (x, y) becomes $(x, y, 1)$; now translation is a *matrix multiply*.
- Bonus: it lets us express **points at infinity** $(x, y, 0)$ — the vanishing points.

Math

Affine as one 3×3 multiply: $\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$

Points at infinity & vanishing points

- Set $w = 0$: $(x, y, 0)$ is a **point at infinity** — a pure *direction*, not a location.
- Two parallel lines share the same direction, so they meet at the same point at infinity.
- A homography can send $w = 0$ to a finite $w' \neq 0$: infinity lands *on-image* as a **vanishing point**.

Key idea

"Parallel rails meeting at the horizon" is literally a point at infinity mapped to a finite pixel.

Chaining transforms is matrix multiplication

- Homogeneous form makes composition trivial: do H_1 then H_2 = apply $H_2 H_1$ (right-to-left).
- Undo a transform with its inverse H^{-1}; warpers use it for inverse mapping.
- So "rotate about a point c" = translate by $-c$, rotate, translate back: $T_c R T_{-c}$.

Math

$H_{\text{about } c} = T(c)\, R(\theta)\, T(-c)$ — three simple 3×3 matrices multiplied into one.

Good practice

Precompute the product once; then a single multiply warps every pixel.

Scale invariance: $(x, y, w) \equiv (\lambda x, \lambda y, \lambda w)$

- A homogeneous point is defined only *up to a nonzero scale*: $(2, 4, 2)$ and $(1, 2, 1)$ are the same point.
- To read off the pixel, **normalise**: divide by the last coordinate $\rightarrow (x/w,\ y/w)$.
- This single "divide" is where perspective will hide (next section).

✔ Good practice

Always normalise before plotting: a returned (x', y', w') with $w' \neq 1$ is not yet a pixel.

The homography

The homography

- A **homography** H is a 3×3 matrix relating two views of a *planar* surface.
- 8 degrees of freedom (the 9th is a scale). Maps straight lines to straight lines, but parallel lines may converge.
- Exactly the model for photographing a flat page/meter from any angle.

Math

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \quad H = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{pmatrix}, \text{ then divide by } w'.$$

Why divide by w'? (the perspective part)

- That final division is what makes far things smaller and parallel lines converge.
- Affine has bottom row $(0\ 0\ 1) \Rightarrow w' = 1$ always $\Rightarrow$ no perspective.
- Homography's bottom row $(h_{31}\ h_{32}\ 1)$ makes w' depend on position $\Rightarrow$ perspective.

Key idea

The nonlinear "divide by w'" is the whole difference between affine and perspective.

Worked example: one point through H

Math

$$H = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.001 & 0 & 1 \end{pmatrix}$$

Push $(x, y) = (400, 300)$:

$$\begin{aligned} w' &= 0.001(400) + 1 = 1.4 \\ x' &= 400, \quad y' = 300 \\ \text{pixel} &= (400/1.4,\ 300/1.4) \\ &\approx (286,\ 214) \end{aligned}$$

- Only $h_{31} = 0.001$ is nonzero, yet the point moved inward.
- Points with larger x get a larger w', so they shrink more $\Rightarrow$ a wall recedes.
- Set $h_{31} = h_{32} = 0$ and it collapses back to a pure affine map.

How many points define a homography?

- 8 unknowns $\Rightarrow$ need 8 equations $\Rightarrow$ **4 point correspondences** (each gives 2).
- `cv2.getPerspectiveTransform(4 src, 4 dst)` for exactly 4 points.
- With many noisy matches: `cv2.findHomography(pts, pts, cv2.RANSAC)` (next session).

✔ Good practice

4 good corners is all you need. For DocScan, those 4 are the document's corners.

Setting up the equations (a peek at the DLT)

- Each correspondence $(x, y) \to (x', y')$ expands the ratios and clears the denominator w'.
- That yields *two* linear equations in the 8 unknowns per point.

Math

From $x' = \dfrac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}$, multiply up:

$$h_{11}x + h_{12}y + h_{13} - x'(h_{31}x + h_{32}y) = x'$$

Four points $\Rightarrow$ 8 such rows $\Rightarrow$ solve $A\mathbf{h} = \mathbf{b}$ (or SVD for the noisy case).

Note

OpenCV does all of this inside `getPerspectiveTransform`; the point is to know it is just linear algebra.

In-class exercise

In-class exercise (10 min)

Reason about the models

1. Which is the simplest transform that can rotate *and* uniformly enlarge a logo? How many DoF?
2. Write the homogeneous 3×3 matrix for a pure translation of $(+30, -10)$.
3. A homography returns $(x', y', w') = (600, 900, 3)$. What pixel is that?
4. Why can an affine transform never make two parallel rails meet at the horizon?

Debrief

- (1) A **similarity** (rotation + uniform scale + translation): 4 DoF.
- (2) $\begin{pmatrix} 1 & 0 & 30 \\ 0 & 1 & -10 \\ 0 & 0 & 1 \end{pmatrix}$.
- (3) Normalise: $(600/3,\ 900/3) = (200,\ 300)$.
- (4) Its bottom row is (0 0 1), so $w' \equiv 1$: no per-point division, hence no convergence.

Key idea

Everything reduces to: build a matrix, multiply, divide by w'. The model choice sets what stays fixed.

Pitfalls & good practice

Common pitfalls with the models

✗ Using affine where you need projective

A page tilted away from the camera has converging edges. `warpAffine` will straighten rotation but leave a keystone — the far edge stays too small.

✗ Forgetting to normalise

Plotting (x', y') without dividing by w' puts points wildly off-image. Always divide.

Good practice

✔ Pick the least powerful model that fits

More DoF = more freedom to overfit noise. Use rigid/affine when the scene really is fronto-parallel; reserve the homography for genuine perspective.

✔ Keep everything in homogeneous form

Chaining transforms is just matrix products $H_2 H_1$. Convert to pixels only at the very end.

Wrap-up

What you should now be able to do

- Place a transform in the hierarchy (rigid $\rightarrow$ similarity $\rightarrow$ affine $\rightarrow$ projective) and count its DoF.
- Write rigid, affine, and projective maps in homogeneous 3×3 form.
- Explain *why* the divide-by-w' step is the entire source of perspective.
- Argue why 4 correspondences (8 equations) pin down a homography.

Next session

S2 — from model to scanner

We turn the homography into the **DocScan** pipeline:

- order the 4 corners and pick a destination rectangle,
- `warpPerspective` with the right **interpolation**,
- and align *automatically* from feature matches using **RANSAC**.

Come with today's mental model: build H, multiply, divide by w'.

A transform is just a matrix and a division.

Choose what stays fixed; the homography fixes only straight lines — enough to model a camera.